

Practical Task

A.Shashank Reddy - 2520090139

TASK-1:

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution

Creating file:

```
shashankreddy@Shashanks-MacBook-Air ~ % nano task.c
```

```
shashankreddy@Shashanks-MacBook-Air ~ %
```

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main()
{
    pid_t pid;

    printf("=====\n");
    printf(" PROCESS CREATION USING FORK\n");
    printf("=====\n");

    printf("\nBefore fork():\n");
    printf("PID = %d\n", getpid());
    printf("PPID = %d\n", getppid());

    pid = fork();

    if (pid < 0)
```

```

{
    printf("Fork failed!\n");
    return 1;
}
else if (pid == 0)
{
    printf("\n\n");
    printf("-----\n");
    printf("    CHILD PROCESS\n");
    printf("-----\n");
    printf("Child PID = %d\n", getpid());
    printf("Child PPID = %d\n", getppid());
    printf("Child is running...\n");
    sleep(3);
    printf("Child process is terminating...\n");
    exit(0);
}
else
{
    printf("\n\n");
    printf("-----\n");
    printf("    PARENT PROCESS\n");
    printf("-----\n");
    printf("Parent PID = %d\n", getpid());
    printf("Parent PPID = %d\n", getppid());
    printf("Child PID = %d\n", pid);
    printf("Parent is waiting for child...\n");
    wait(NULL);
    printf("Child process has terminated.\n");
    printf("Parent process is terminating...\n");
}
}

```

```
return 0;
}
```

Output:

```
=====
PROCESS CREATION USING FORK
=====

Before
fork(): PID =
16213
PPID = 315
-----

PARENT PROCESS
-----

Parent PID   =
16213 Parent PPID
              = 315
Child PID    =
16214
Parent is waiting for
child...
-----

CHILD PROCESS
-----

Child PID = 16214
Child PPID = 16213
Child is running...

Child process is terminating...
Child process has terminated.

Parent process is terminating...
```

TASK-2

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Creating file:

```
shashankreddy@Shashanks-MacBook-Air ~ % nano shashank.c
```

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    printf("=====\n");
    printf("  PROCESS STATE EXPERIMENT\n");
    printf("=====\n");

    pid =
    fork(); if
    (pid < 0)
    {
        perror("fork");
        return 1;
    }
    if (pid == 0)
    {
        printf("\n[CHILD] Process created\n");
        printf("[CHILD] PID = %d\n", getpid());
        printf("[CHILD] PPID = %d\n", getppid());
        printf("\n[CHILD] STATE: RUNNING\n");
        printf("[CHILD] Performing work...\n");
        for (long long i = 0; i < 3000000000LL; i++)
        {
```

```
    if (i % 1000000000LL == 0)
    {
        printf("[CHILD] Working... %lld\n", i);
    }
}

printf("\n[CHILD] STATE: WAITING/SLEEPING\n");
sleep(10);
printf("\n[CHILD] STATE: RUNNING AGAIN\n");
printf("[CHILD] Terminating...\n");
exit(0);
}
else
{
    printf("\n[PARENT] Parent process created\n");
    printf("[PARENT] PID   = %d\n", getpid());
    printf("[PARENT] Child PID = %d\n", pid);
    printf("\n[PARENT] Monitor the child using:\n");
    printf("ps -p %d -o pid,ppid,state,cmd\n", pid);
    printf("\n[PARENT] Waiting for child to terminate...\n");
    wait(NULL);
    printf("\n[PARENT] Child has terminated.\n");
    printf("[PARENT] Parent process terminating.\n");
}
return 0;
}
```

Compiling:

```
root@LAPTOP-4H877K4U:~# nano ptask2.c
root@LAPTOP-4H877K4U:~# gcc ptask2.c -o ptask2
root@LAPTOP-4H877K4U:~# ./ptask2
```

Output:

```
=====
PROCESS STATE EXPERIMENT
=====

[PARENT] Parent process created
[PARENT] PID    = 16292
[PARENT] Child PID = 16293
[PARENT] Monitor the child using:
ps -p 16293 -o pid,ppid,state,cmd
[PARENT] Waiting for child to terminate...

[CHILD] Process created
[CHILD] PID  = 16293
[CHILD] PPID = 16292
[CHILD] STATE: RUNNING
[CHILD] Performing work...
[CHILD] Working... 0
[CHILD] Working... 1000000000
[CHILD] Working... 2000000000
[CHILD] STATE: WAITING/SLEEPING
```

```
[CHILD] STATE: RUNNING AGAIN
[CHILD] Terminating...
[PARENT] Child has terminated.
[PARENT] Parent process terminating.
```